AI ASSISTED CODING LAB 4

Lab Experiment: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques

Name : G.Sreeja

Roll No : 2503A52692

Batch No : 24

Question 1: Zero-Shot Prompting (Customer Email Classification)

Prepare five short sample emails belonging to Billing, Technical Support, Feedback, or Others. Write a zero-shot prompt, without examples, to classify an email.

Sample Data:

1. “I was charged twice for my monthly subscription.” — Billing
2. “The application crashes whenever I try to log in.” — Technical Support
3. “Your new update is easy to use and very helpful.” — Feedback
4. “Please tell me your office address.” — Others
5. “My refund has not appeared on my bank statement.” — Billing

Prompt:

> Classify the customer email into exactly one category: Billing, Technical Support, Feedback, or Others. Do not provide examples. Return only the category name. Email: “The application crashes whenever I try to log in.”

Code:

```
def classify_email(email):
    text = email.lower()

    billing_words = ["charged", "bill", "payment", "refund", "invoice", "subscription"]
    support_words = ["crash", "error", "not working", "cannot", "can't", "login", "log in", "bug"]
    feedback_words = ["helpful", "easy to use", "suggest", "feedback", "excellent", "improve"]

    if any(word in text for word in billing_words):
        return "Billing"
    if any(word in text for word in support_words):
        return "Technical Support"
    if any(word in text for word in feedback_words):
        return "Feedback"
    return "Others"
```

```
email = "The application crashes whenever I try to log in."
print(classify_email(email))
```

Output:

```
Technical Support
```

Observation:

The prompt gives category definitions through labels only and includes no labeled examples, so it represents zero-shot prompting. The keyword-based Python code is a simple demonstration; ambiguous emails may need better rules or an LLM.

Question 2: One-Shot Prompting (Intent Classification for Chatbot Queries)

Create six short chatbot queries mapped to Account Issue, Order Status, Product Inquiry, or General Question. Use one labeled example before classifying a new query.

Sample Data:

1. “I forgot my password.” — Account Issue
2. “Where is my parcel?” — Order Status
3. “Does this laptop support Bluetooth?” — Product Inquiry
4. “What time does customer service open?” — General Question
5. “Please unlock my account.” — Account Issue
6. “Has my order been shipped?” — Order Status

Prompt:

> Classify the query into Account Issue, Order Status, Product Inquiry, or General Question. Example: Query: “Where is my parcel?” Intent: Order Status. Now classify: “Does this laptop support Bluetooth?” Return only the intent.

Code:

```
def classify_intent(query):
    text = query.lower()

    if any(word in text for word in ["password", "account", "locked", "login", "log in"]):
        return "Account Issue"
    if any(word in text for word in ["order", "parcel", "delivery", "shipped", "tracking"]):
        return "Order Status"
    if any(word in text for word in ["product", "laptop", "phone", "bluetooth", "features", "support"]):
        return "Product Inquiry"
    return "General Question"

query = "Does this laptop support Bluetooth?"
print(classify_intent(query))
```

Output:

```
Product Inquiry
```

Observation:

One labeled example demonstrates the desired input-output format. The code uses simple keyword rules to show the classification; in real use, more context may be needed for overlapping categories.

Question 3: Few-Shot Prompting (Student Feedback Sentiment Analysis)

Classify student feedback as Positive, Negative, or Neutral. Use multiple labeled examples to guide the response.

Prompt:

```
Classify feedback as Positive, Negative, or Neutral. Examples: “The lectures are excellent.” → Positive; “The explanations are confusing.” → Negative; “The class starts at 9 AM.” → Neutral; “I enjoyed the practical sessions.” → Positive. Now classify: “The assignment instructions were unclear.” Return only the label.
```

Code:

```
def classify_sentiment(feedback):
    text = feedback.lower()
    positive_words = ["excellent", "good", "helpful", "enjoyed", "clear", "great", "useful"]
    negative_words = ["bad", "confusing", "unclear", "poor", "difficult", "disappointing"]

    pos = sum(word in text for word in positive_words)
    neg = sum(word in text for word in negative_words)

    if pos > neg:
        return "Positive"
    if neg > pos:
        return "Negative"
    return "Neutral"

feedback = "The assignment instructions were unclear."
print(classify_sentiment(feedback))
```

Output:

```
Negative
```

Observation:

Few-shot examples show the model how each sentiment label should be applied. They can improve consistency, but the sample set should include varied wording to reduce misclassification.

Question 4: Few-Shot Prompting (Course Recommendation Level)

Classify learner queries into Beginner, Intermediate, or Advanced levels.

Prompt:

> Classify the learner query as Beginner, Intermediate, or Advanced. Examples: “I am new to Python and want to learn variables.” → Beginner; “I know loops and functions and want to build a small project.” → Intermediate; “I want to optimize a neural network and deploy it at scale.” → Advanced. Now classify: “I understand Python basics and want to learn object-oriented programming.” Return only the level.

Code:

```
def classify_level(query):
    text = query.lower()
    advanced_words = ["optimize", "deploy", "at scale", "deep learning", "neural network", "distributed"]
    intermediate_words = ["project", "object-oriented", "oop", "api", "database", "framework"]
    beginner_words = ["new to", "beginner", "basics", "start learning", "variables"]

    if any(word in text for word in advanced_words):
        return "Advanced"
    if any(word in text for word in intermediate_words):
        return "Intermediate"
    if any(word in text for word in beginner_words):
        return "Beginner"
    return "Intermediate"

query = "I understand Python basics and want to learn object-oriented programming."
print(classify_level(query))
```

Output:

```
Intermediate
```

Observation:

Few-shot examples make the level boundaries clearer and help match queries to suitable course difficulty. The rules are illustrative; actual recommendations should consider the learner's full skills and goals.

Question 5: Few-Shot Prompting (Social Media Post Moderation)

Classify social media posts as Acceptable, Offensive, or Spam.

Prompt:

> Classify each post as Acceptable, Offensive, or Spam. Examples: “I enjoyed the event today.” → Acceptable; “You are worthless and should be ashamed.” → Offensive; “Click this link now to claim a free prize!!!” → Spam. Now classify: “Join our page for repeated offers; click this link to win free gifts.” Return only the label.

Code:

```
def moderate_post(post):
    text = post.lower()
    spam_words = ["click this link", "free prize", "win free", "claim", "limited offer", "buy now"]
    offensive_words = ["worthless", "stupid", "idiot", "hate you", "should be ashamed"]

    if any(word in text for word in spam_words):
        return "Spam"
    if any(word in text for word in offensive_words):
        return "Offensive"
    return "Acceptable"

post = "Join our page for repeated offers; click this link to win free gifts."
print(moderate_post(post))
```

Output:

```
Spam
```

Observation:

Examples clarify the distinction between normal, offensive, and promotional content. Zero-shot moderation can struggle with sarcasm, context, and indirect abuse; keyword rules are only a basic demonstration and should not be treated as a reliable moderation system.

Conclusion

This lab explored zero-shot, one-shot, and few-shot prompting through customer email classification, chatbot intent classification, student feedback sentiment analysis, course-level classification, and social media post moderation. Zero-shot prompts rely on instructions alone, one-shot prompts provide one labeled example, and few-shot prompts provide several examples. Examples can improve output consistency and clarify category boundaries, but results still depend on the quality and variety of examples. The Python programs included here demonstrate the classification logic in a simple way; they are not actual large language models and may misclassify ambiguous text.

Note: Code sections are formatted to resemble the dark VS Code editor. Run each program in VS Code to capture actual execution screenshots if required.